Basic Notations and Terminologies used in FAFL

2.1 Alphabet

- An alphabet is a finite, non-empty set of symbols.
- A language consists of various symbols from which the words, statements can be obtained.
- The symbols are called alphabets.

Examples :

(i) The alphabet of C language has the letters from 'a' to 'z', digits from '0' to '9' symbols.

$\Sigma = \{ a, b, \ldots, z, A, B, \ldots, Z, *, -, +, \%, \ldots \}$

(ii) The machine language is made up of only zero's and one's, so alphabet of machine language is denoted by,

$\Sigma = \{ 0, 1 \}$

→ Write 5 different alphabets used in Computer Science.

* Binary alphabet → $\Sigma = \{0,1\}$
* Decimal digits → $\Sigma = \{0,1,2,3,4,5,6,7,8,9\}$
* Hexadecimal symbol → $\Sigma = \{0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F\}$
* Arithmetic operator alphabets → $\Sigma = \{ +, -, \times, \div \}$
* Boolean alphabets → $\Sigma = \{0,1\}$ where, 0 = False, 1 = True
* Logical operator alphabets → $\Sigma = \{$ AND, OR, NOT $\}$

2.2 String

- A sequence of symbols obtained from the alphabets of a language is called string.

Formally,

- A string is a finite sequence of symbols taken from an alphabet.

Note : (i) Sequence of symbols

$\Sigma = \{ a, b, c \}$

Valid Strings :

ab, ba, bc, abc, abba, ...

Invalid Strings :

ab d, a1, 0abc, ...

Empty String :

- An empty string is denoted by the symbol (epsilon) ϵ and λ (lambda).
- A string containing no symbol is called empty string.

Example : Let $\Sigma = \{0,1\}$ is a set of alphabets. The various strings that can be obtained from Σ are,

$\Sigma = \{ \epsilon, 0, 1, 00, 11, 000, 1111, 10010, \ldots \}$ ← Valid strings

- $\Sigma = \{0, 1, 01, 22, \ldots\}$ ← Invalid string

Example : $\Sigma = \{a,b,c\}$ is a set of alphabets.

$\Sigma = \{ a, b, c, aa, bb, abc, abbc, bbaacc, \ldots \}$ ← Valid string

$\Sigma = \{ 0, 1, 2, 0, 2, \ldots \}$ ← Invalid string

Basic Notations and Terminologies used in FAFL

2.3 Length of a String

- The length of a string u is the number of symbols in u and is denoted by $|u|$ (modulus).
- If $u = a_1 a_2 a_3 \dots a_n$, then the length of u is $|u| = n$.

Example :

If $u = abbab$, then $|u| = 5$.

Note : Lowercase letters such as a, b, u, w, x, y are normally used to indicate strings.

Example : $u = 1010111$

2.4 Concatenation of Two Strings

- The concatenation of two strings u and v is the string obtained by writing the letters of string u followed by the letters of string v.
- If $u = a_1 a_2 \dots a_m$ and $v = b_1 b_2 \dots b_n$, then
  $uv = a_1 a_2 \dots a_m b_1 b_2 \dots b_n$.

Example :

u = Computer
v = Science
uv = ComputerScience

Note : Concatenation with Empty String

Let $w = abc$. Then,

$w\varepsilon = abc$ and $\varepsilon w = abc$.

$$w\varepsilon = \varepsilon w = w$$

Basic Notations and Terminologies used in FAFL

2.5 Substrings, Prefix and Suffix

- Let w be a string obtained from the symbols in alphabet Σ. The string w can be decomposed into three strings x, y and z such that

$$w = x\,y\,z$$

x is a prefix
y is a substring
z is a suffix
} parts of string w

- A substring is a contiguous sequence of symbols occurring within a string.

Example :

Let w = abcde.

Valid substrings : a, b, c, ab, bc, cd, de

Invalid substring : ace (not contiguous)

Prefix : A substring that appears at the beginning of a string.

Example :

For w = abcde,

Prefixes : ε, a, ab, abc, abcd, abcde

Suffix : A substring that appears at the end of a string.

Example :

For w = abcde,

Suffixes : ε, e, de, cde, bcde, abcde

Basic Notations and Terminologies used in FAFL

Prefix :

- A prefix of a string is obtained by taking zero or more symbols from the beginning (left side) of a string.

Example : $w = abcde$

ε
a
ab
abc
abcd
abcde

$w = Rama$

ε
R
Ra
Ram
Rama

Suffix :

- A suffix of a string is obtained by taking zero or more symbols from the end (right side) of a string.

Example : $w = abcde$

ε
e
de
cde
bcde
abcde

$w = Rama$

ε
a
ma
ama
Rama

→ w = Computer : write substring, prefix, suffix.

- **Substring :** (any continuous part)

  w = Computer

  C, o, m, pu, t, e, r, co, om, mp, pu, ut, te, er, com, omp, put, tev, comp, put, uter, Computer

- **Prefix :** (Beginning)

  w = Computer

Σ	Comput
ε	Compute
C	Computer
Co	
Com	
Comp	
Compu	

Basic Notations and Terminologies used in FAFL

- **Suffix :** (Ending)

  w = Computer

Σ
r
er
ter
uter
puter
mputer
omputer
Computer

Reversal of a String :

- The reversal of a string is the string written with the symbols in reverse order.

  Let $u = a_1 a_2 a_3 \cdots a_n$

  then, the reversal of u is denoted by u^R (read as u reverse)

  So, $u^R = a_n, a_{n-1}, a_{n-2}, \cdots, a_3, a_2, a_1$.

Example :

u = Computer Science

u^R = ecneicS retupmoC

Properties of Reversal :

* $(\varepsilon)^R = \varepsilon$
  - ↳ The reversal of the empty string ε is ε.
* $(w)^{R\,R} = w$
  - ↳ The single symbol remains unchanged.
* $(w^R)^R = w$
  - ↳ Reverse twice gives the original string.

  Eg: $s = abc$; $s^R = cba$; $(s^R)^R = abc$.
* $(xy)^R = y^R x^R$

Basic Notations and Terminologies used in FAFL

Power of an Alphabet

- The power of an alphabet (Σ^i) is the set of words of length i.

Example :

1) Let $\Sigma = \{0, 1\}$, then

$\Sigma^0 = \{\varepsilon\}$

$\Sigma^1 = \{0, 1\}$

$\Sigma^2 = \{01, 10, 11, 00\}$

$\Sigma^3 = \{000, 111, 101, 010, 011, \cdots\}$

2) Let $\Sigma = \{a, b, c\}$ Σ has one symbol of the alphabet, not two separate symbols), then,

$\Sigma^0 = \{\varepsilon\}$

$\Sigma^1 = \{a, b, c\}$

$\Sigma^2 = \{aa, ab, ac, ba, bb, bc, ca, cb, cc\}$ → Because, aa, ab, ac ba, bb, bc ca, cb, cc (total 9 strings)

$\Sigma^3 = \{aaa, aab, aac, aba, abc, acb, bca, bcb, cbc\}$

Note :

- Number of strings of length n over an alphabet Σ is $|\Sigma^n| = m^n$

where, m = alphabet size (number of symbols)
n = length of string.

$2^4 = 16$ [if length is 4]

3) Let $\Sigma = \{abc, abc\}$

$\Sigma^0 = \{\varepsilon\}$

$\Sigma^1 = \{abc\}$

$\Sigma^2 = \{abc, abc\}$

$\Sigma^3 = \{abc, abc, abc\}$

Basic Notations and Terminologies used in FAFL

4) Let $\Sigma = \{a, b, c, d, e\}$, then

$\Sigma^0 = \{\varepsilon\}$

$\Sigma^1 = \{a, b, c, d, e\}$

$\Sigma^2 = \{ab, ac, ad, ae, ba, bc, bd, be, ca, cb, cd, ce, da, db, dc, de, ea, eb, ec, ed, ee\}$

$\Sigma^3 = \{abc, abd, abe, acb, acd, ace, adb, adc, ade, aeb, aec, aed, \cdots, edc, ede, ecd, ece, ecb\}$

Variation of Power of an Alphabet

- It has two types :
  - * Kleene closure (*)
  - * Kleene plus (+)

Kleene Closure :

- Kleene closure of an alphabet is the set of all possible strings (including ε) that can be formed using the symbols of an alphabet.
- The kleene closure is defined as follows,

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \cdots\cdots$$

Example :

- Let $\Sigma = \{a, b\}$

$\Sigma^0 = \{\varepsilon\}$

$\Sigma^1 = \{a, b\}$

$\Sigma^2 = \{aa, ab, ba, bb\}$

$\therefore \Sigma^* = \{\varepsilon, a, b, ab, ba, aa, bb\}$

Kleene Plus :

- The kleene plus, denoted by Σ^+, is defined as follows,

$$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \cdots\cdots$$

Example :

- Let $\Sigma = \{0, 1\}$

2.3 Language

- A language can be defined as a set of strings obtained from Σ.
- In other words, a language is a subset of Σ^*.

Mathematically,

$$L \subseteq \Sigma^*$$

where,

L : Language
Σ : Alphabet
Σ^* : Kleene closure, set of all possible strings over Σ
$\subseteq$: is a subset of

Example :

1) A language of strings consisting of equal number of 0's and 1's can be represented as,

$$L = \{ 0, 1, 01, 10, 0011, 0101, 1010, 1100 \}$$

(001 is not valid because it has 2 zeros and 1 one)

2) The language of strings consisting of 'n' numbers of 0's followed by 'n' numbers of 1's can be represented using set as shown below :

$$L = \{ 0^n 1^n \mid n \geq 0 \}$$

This means all the 0's come first, followed by all the 1's.

Basic Notations and Terminologies used in FAFL

Example (continued) :

Let $\Sigma = \{0, 1\}$

$n = 0 \Rightarrow \epsilon$
$n = 1 \Rightarrow 01$
$n = 2 \Rightarrow 0011$
$n = 3 \Rightarrow 000111$
$n = 4 \Rightarrow 00001111$
$n = 5 \Rightarrow 0000011111$

All the strings must have first n number of 0's, followed by n number of 1's.

3) A language containing the empty string ϵ (epsilon) is denoted by

$$L = \{ \epsilon \}$$

This language has only one string in the set, so the string is ϵ.

4) Empty language is denoted by

$$\phi$$

What is Sentence ?

- What is Sentence ? Give an example ?
- A string that belongs to a language is called a word or sentence of the language.

Example :

Let $L = \{ \epsilon, 0, 001, 00011 \}$

- Each symbol in L, separated by comma, is a sentence of the language L.

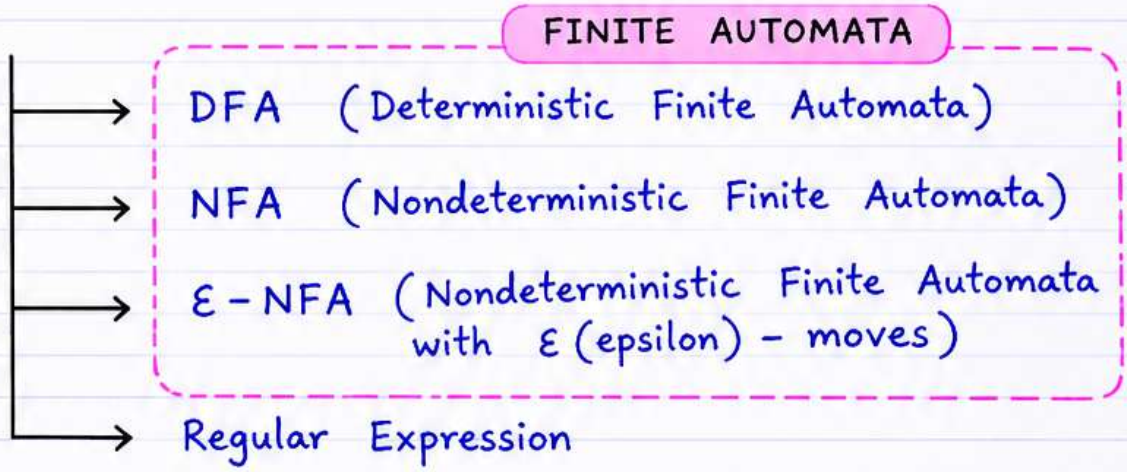
Types of Languages in FLAT

Types of Languages



➤ What are the different ways of describing the regular languages (Type 3) ?

- The regular languages can be described using 4 different methods.



➤ What are Regular Languages ?

- A regular language is a language that can be recognized (accepted) using a finite automata (DFA, NFA, ε-NFA) and described using a regular expression.

Note :

- Type 0 : Phrase structured language (Unrestricted grammar)
- Type 1 : Context sensitive language (CSL)
- Type 2 : Context free language (CFL)
- Type 3 : Regular language (RL)

1.3 Finite Automata

Definition :

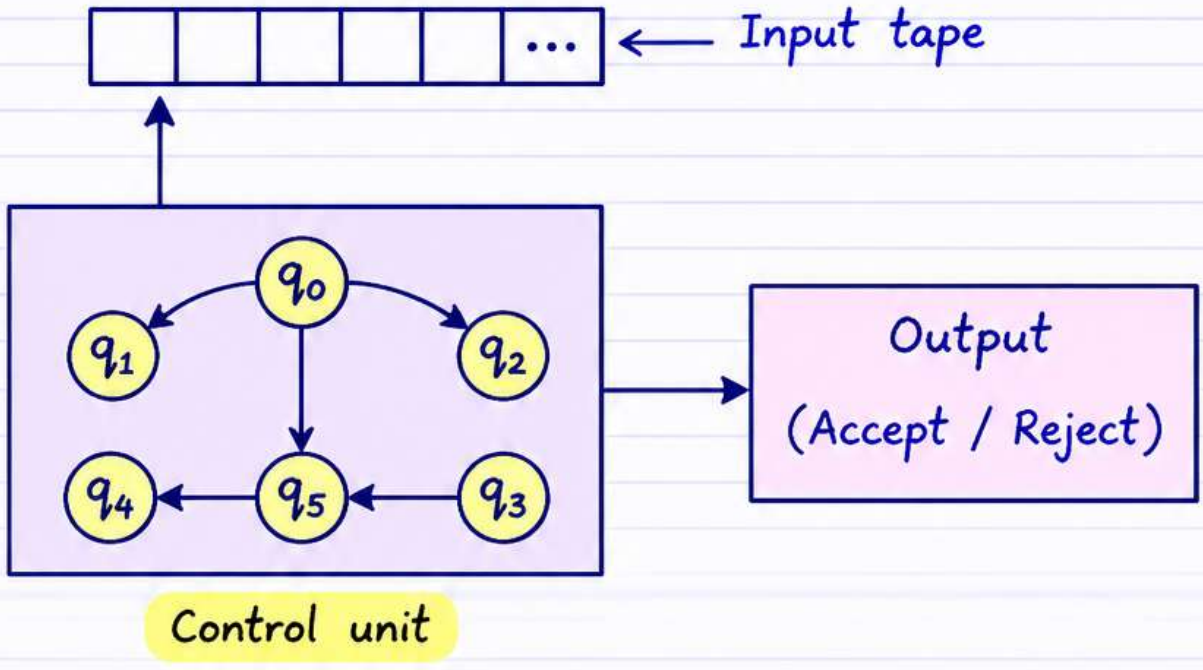
- A finite automaton is a simple mathematical machine that reads an input string one symbol at a time and finally decides whether to accept or reject the string.

Example :

- Suppose the language contains only AB.
  - Input : AB — machine says ACCEPTED.
  - Input : AC — machine says REJECTED.

Components of Finite Automata (FA)

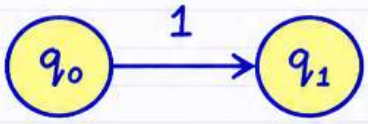
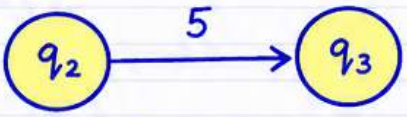
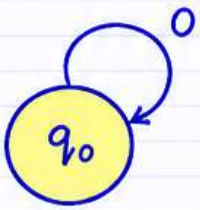
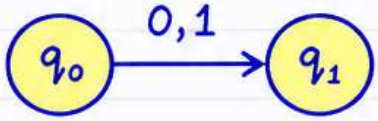
- 1. Input tape
  2. Control unit
  3. Output



Basic structure of a finite automaton

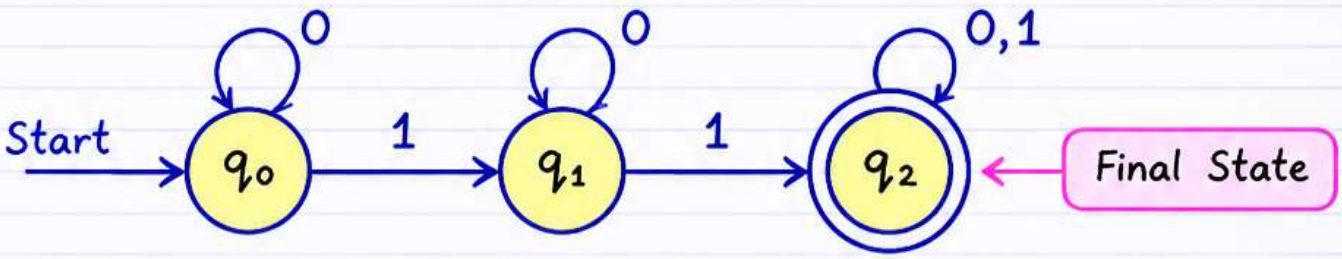
Symbols and Notations Used in FA

- The following symbols are commonly used while designing Finite Automata :

Symbol / Diagram	Meaning
	• Two circles are used to represent a final state. Here, q_0 is the final state.
	• Input 1 goes from q_0 to q_1. This indicates there is a transition from state q_0 on input symbol 1 to state q_1. • This is represented as $\delta(q_0, 1) = q_1$. Example :  $\delta(q_2, 5) = q_3$.
	• A label '0' starts and ends in q_0. This indicates that the machine in state q_0, on reading '0', remains in the same state q_0. • This is represented as $\delta(q_0, 0) = q_0$.
	• An arrow with 0, 1 goes from state q_0 to q_1. This indicates that the machine in state q_0, on reading a 0 or a 1, enters q_1. • This is represented as $\delta(q_0, 0) = q_1$ $\delta(q_0, 1) = q_1$.

1.4 Deterministic Finite Automata (DFA)

- **Pictorial Representation of DFA :**



Alphabet : $\Sigma = \{0, 1\}$ **State Set :** $Q = \{q_0, q_1, q_2\}$

- **Transition Values :**

$$\delta(q_0, 0) = q_0$$
$$\delta(q_0, 1) = q_1$$
$$\delta(q_1, 0) = q_1$$
$$\delta(q_1, 1) = q_2$$
$$\delta(q_2, 0) = q_2$$
$$\delta(q_2, 1) = q_2$$

- **Transition Function :**

$$\delta : (Q \times \Sigma) \longrightarrow Q$$

- The transition function maps a state and an input symbol to the next state.

What is a Transition Function ?

- A transition function δ is a function from $Q \times \Sigma$ to Q. It specifies the next state of the automaton for each current state and input symbol.

Example :

$Q = \{q_0, q_1, q_2\}$, $\Sigma = \{0, 1\}$

Where,

- Q : Set of states.
- Σ : Input alphabet.
- $Q \times \Sigma$: Every possible combination of a state and input symbol.

Example :

- $\Sigma = \{0, 1\}$
- $Q = \{q_0, q_1, q_2\}$
- $Q \times \Sigma = \{(q_0,0), (q_0,1), (q_1,0), (q_1,1), (q_2,0), (q_2,1)\}$

(Cross product)

All possible transitions for a DFA

- The cross product is used to list all possible transitions, thus the transition function will be complete.

What is DFA ?

- The DFA is a five tuple or quintuple indicating five components.

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where,

- M : is the name of the machine; it can also be called by any name.
- Q : is a non-empty, finite set of states.
- Σ : is a non-empty, finite set of input alphabets.
- δ : $(Q \times \Sigma) \rightarrow Q$ is the transition function, which maps from $Q \times \Sigma$ to Q.
- q_0 : is the start state, where $q_0 \in Q$.
- F : is a subset of Q; it is the final (accepting) state(s), where $F \subseteq Q$.

1.5 Simple Notations for DFA

↣ What are the various simple notations used to specify a DFA ?

⇒ There are two notations :

- Transition diagram (or Transition graph).
- Transition table.

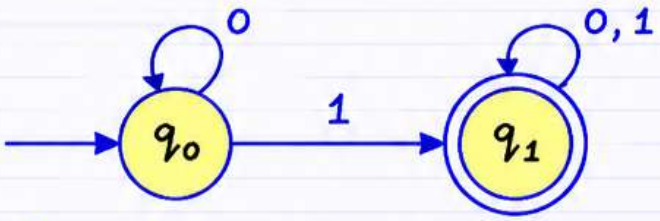
• What is a Transition Diagram ?

A Transition diagram is nothing but a graphical representation of a DFA.

The Transition diagram for DFA,

$M = (Q, \Sigma, \delta, q_0, F)$ is defined as a graph with circles, arrows, arcs with labels and two concentric circles etc.

Example :



Transition Diagram

$\delta(q_0, 0) = q_0$
$\delta(q_0, 1) = q_1$
$\delta(q_1, 0) = q_1$
$\delta(q_1, 1) = q_1$

Transition Function

• What is a Transition Table ?

↣ The Transition table for DFA,

$M = (Q, \Sigma, \delta, q_0, F)$ is defined as a tabular representation of a transition function or diagram.

Example :

State	Input Symbol	
	0	1
→ q_0	q_0	q_1
★ q_1	q_1	q_1

→ : Start state
★ : Final state (accepting state)

1.5 Simple Notations for DFA

- **Transition Table :**
  - The rows of the table represent current states of the DFA obtained from Q.
  - The columns represent input symbols obtained from Σ (sigma).

δ	0	1
$\rightarrow q_0$	q_0	q_1
$\bigstar\ q_1$	q_1	q_1

Legend :

$\rightarrow$: Start state

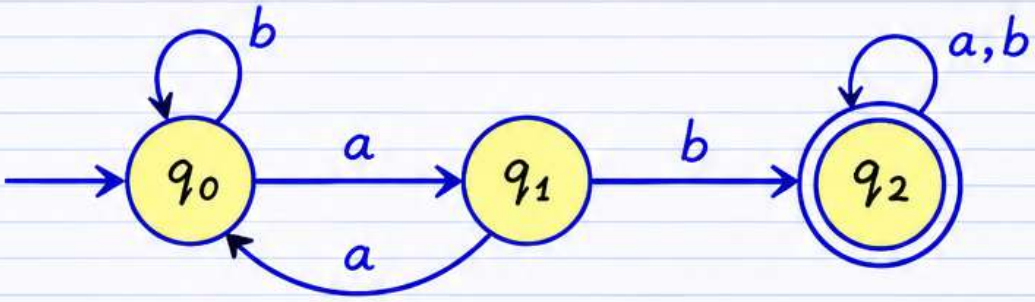
$\bigstar$: Final (accepting) state

Example: Transition Function and Transition Table

- Rows of the table represent the current states of the DFA obtained from Q.
- Columns represent input symbols obtained from Σ (sigma).

δ	a	b
→ q_0	q_1	q_0
q_1	q_0	q_2
★ q_2	q_2	q_2

- Example for Transition Function & Transition Table



$\delta(q_0, a) = q_1$
$\delta(q_0, b) = q_0$
$\delta(q_1, a) = q_0$
$\delta(q_1, b) = q_2$
$\delta(q_2, a) = q_2$
$\delta(q_2, b) = q_2$

Transition Function

δ	a	b
→ q_0	q_1	q_0
q_1	q_0	q_2
★ q_2	q_2	q_2

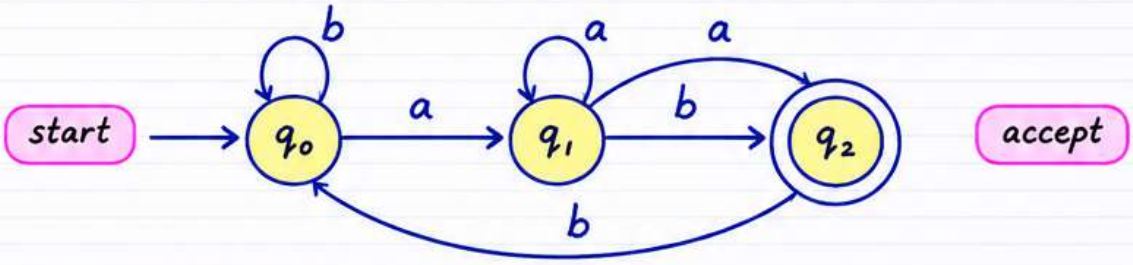
Transition Table

Example : Processing of Strings by a DFA

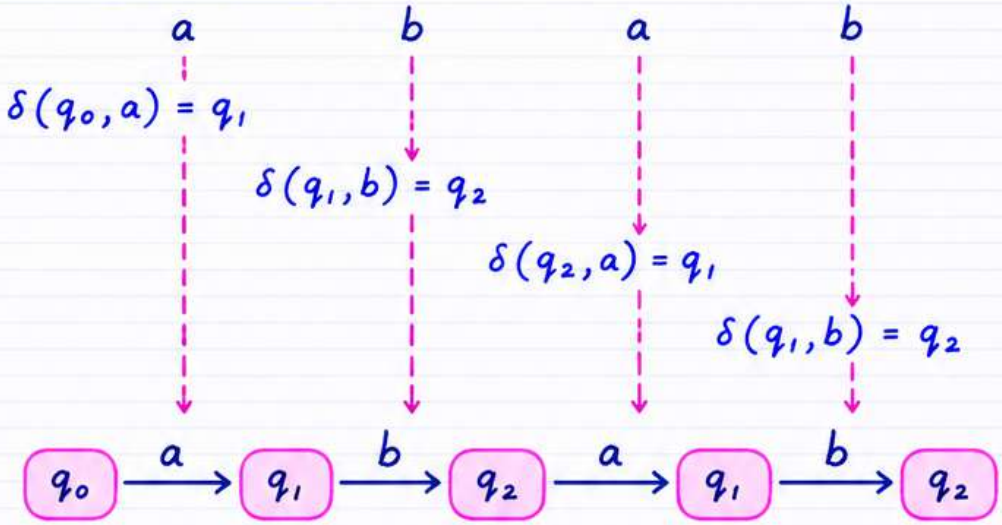
Problem : How a DFA processes the string "abab" and "abb"?

Solution :

- The moves made by the DFA for the string "abab" is shown below.



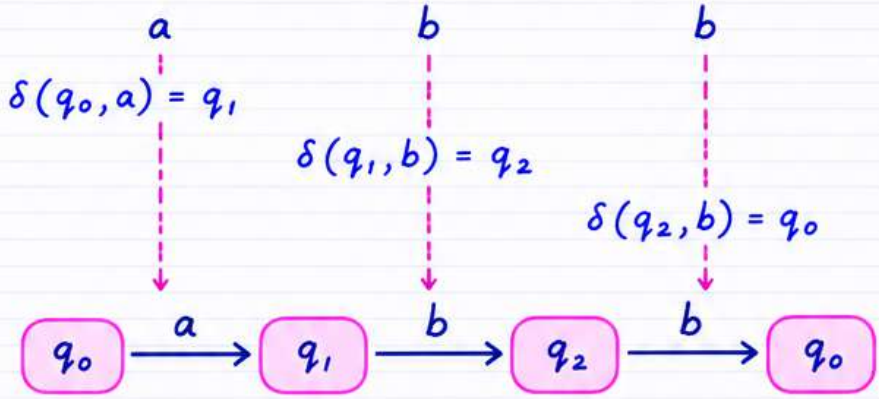
Input String : abab



At the end of the string "abab", the DFA will be in state q_2 which is the final state. So, the string abab is accepted by the machine.

- The moves made by the DFA for the input string "abb" is shown below.

Input String : abb



At the end of the string "abb", the DFA will be in state q_0 which is not a final state. So, the string abb is rejected by the machine.

Example: Processing of Strings by a DFA

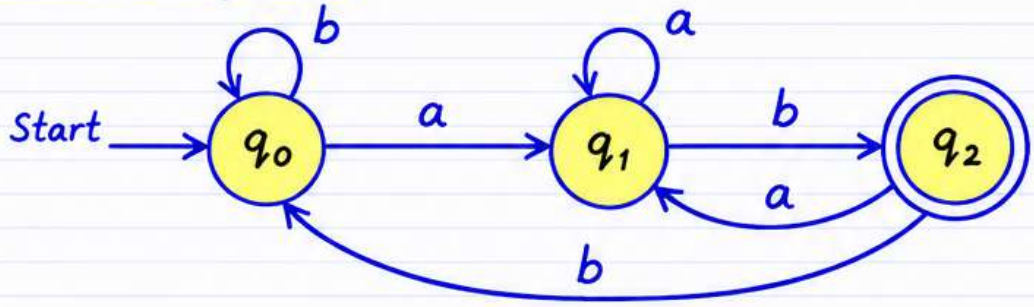
- **DFA :**

Consider the DFA $M = (Q, \Sigma, \delta, q_0, F)$ where,

- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{a, b\}$
- q_0 = start state
- $F = \{q_2\}$ (final state)
- δ : Transition function,

$\delta(q_0, a) = q_1$, $\delta(q_0, b) = q_0$
$\delta(q_0, a) = q_1$, $\delta(q_1, b) = q_2$
$\delta(q_2, a) = q_1$, $\delta(q_2, b) = q_0$

Transition Diagram :



- **Processing the string "abaab" :**
  - **Input String :** abaab (five symbols)
  - **Transition Evaluations :**

$\delta(q_0, a) = q_1$	$\delta(q_1, b) = q_2$	$\delta(q_2, a) = q_1$	$\delta(q_1, a) = q_1$	$\delta(q_1, b) = q_2$

  - **State Path :** $q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_2 \xrightarrow{a} q_1 \xrightarrow{a} q_1 \xrightarrow{b} q_2$
  - **Result :** "abaab" is **ACCEPTED** because q_2 is a final state.

- **Processing the string "abb" :**
  - **Input String :** abb (three symbols)
  - **Transition Evaluations :**

$\delta(q_0, a) = q_1$	$\delta(q_1, b) = q_2$	$\delta(q_2, b) = q_0$

  - **State Path :** $q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_2 \xrightarrow{b} q_0$
  - **Result :** "abb" is **REJECTED** because q_0 is not a final state.

Extended Transition Function of DFA

- **Definition :**

The extended transition function $\delta^* : Q \times \Sigma^* \to Q$ is defined recursively as follows.

- **Base Case :**

$$\delta^*(q, \varepsilon) = q$$

Meaning : If the machine is in state q and reads no input, it remains in state q.

Current state = q; input = ε; final state = q.

- **Recursive Case :**

Let $w = xa$, where a is the last symbol of w and x is the remaining string of w.

Then,

$$\delta^*(q, w) = \delta^*(q, xa) = \delta(\delta^*(q, x), a).$$

Extended Transition Function of DFA – Properties & Example

Properties of Extended Transition Function :

1) $\delta^*(q, \varepsilon) = q.$

2) $\delta^*(q, w) = \delta^*(q, xa) = \delta(\delta^*(q, x), a),$

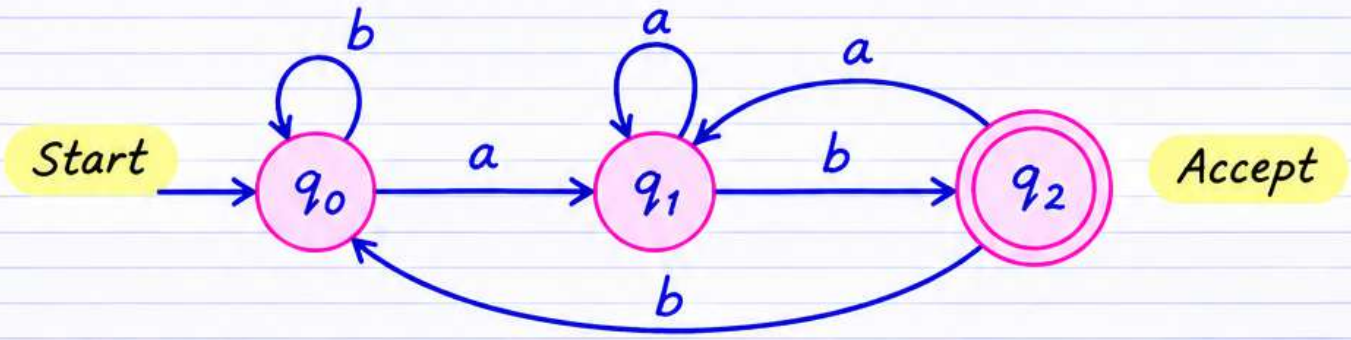
where $w = xa$ and a is the last symbol.

3) $\delta^*(q, aw) = \delta^*(\delta(q, a), w),$

where a is the first symbol and w is the remaining string.

Problem on Extended Transition Function :

Q : What are the moves made by the DFA by processing the string 'abaab' using the extended transition function ?



Solution : Prefixes of abaab

ε

a

ab

aba

abaa

abaab

Extended Transition Function – Step-by-Step Solution

- **Processing the String abaab :**

1) For prefix ε :

$$\delta^*(q_0, \varepsilon) = q_0$$

---

2) For prefix a :

$$\delta^*(q_0, a) = \delta(\delta^*(q_0, \varepsilon), a) = \delta(q_0, a) = q_1$$

---

3) For prefix ab :

$$\delta^*(q_0, ab) = \delta(\delta^*(q_0, a), b) = \delta(q_1, b) = q_2$$

---

4) For prefix aba :

$$\delta^*(q_0, aba) = \delta(\delta^*(q_0, ab), a) = \delta(q_2, a) = q_1$$

---

5) For prefix $abaa$:

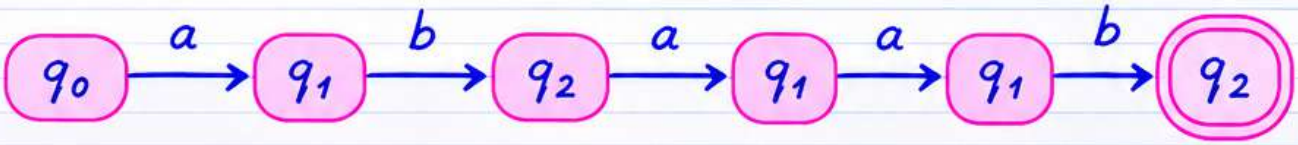
$$\delta^*(q_0, abaa) = \delta(\delta^*(q_0, aba), a) = \delta(q_1, a) = q_1$$

---

6) For prefix $abaab$:

$$\delta^*(q_0, abaab) = \delta(\delta^*(q_0, abaa), b) = \delta(q_1, b) = q_2$$

- **State Path :**



After processing the string abaab, the DFA reaches state q_2, which is a final state. Therefore, the string abaab is **ACCEPTED.**

Language Accepted by a DFA

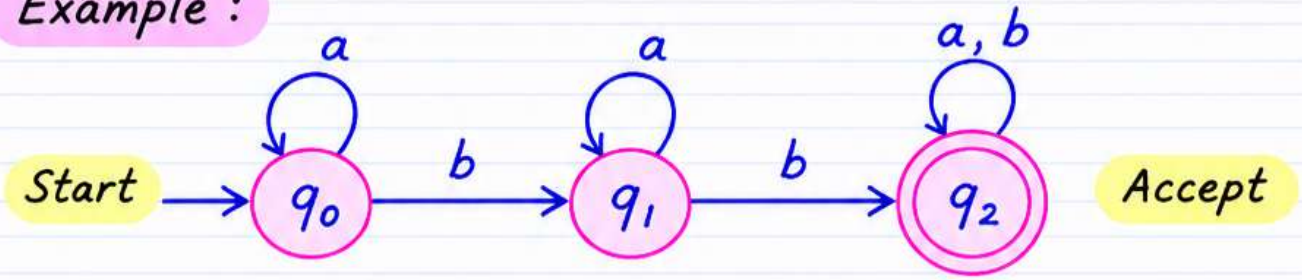
- **Definition :**

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA. A string w is accepted by the machine M if, after processing w from the initial state q_0, the machine reaches a final state. That is, $\delta^*(q_0, w) \in F$.

$$L(M) = \{ w \in \Sigma^* \mid \delta^*(q_0, w) \in F \}$$

$L(M)$ is the language accepted by DFA M.

- **Example :**



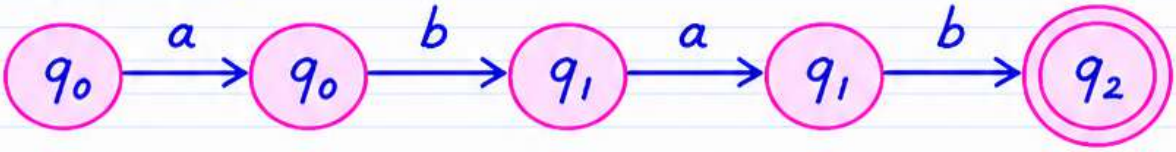
- **Processing the input string abab :**

Input : a b a b

Transitions :

$\delta(q_0, a) = q_0$
$\delta(q_0, b) = q_1$
$\delta(q_1, a) = q_1$
$\delta(q_1, b) = q_2$

State Path :



After processing the string abab, the DFA reaches state q_2, which is a final state. Therefore, abab is **ACCEPTED** by the machine.

Language Rejected by a DFA

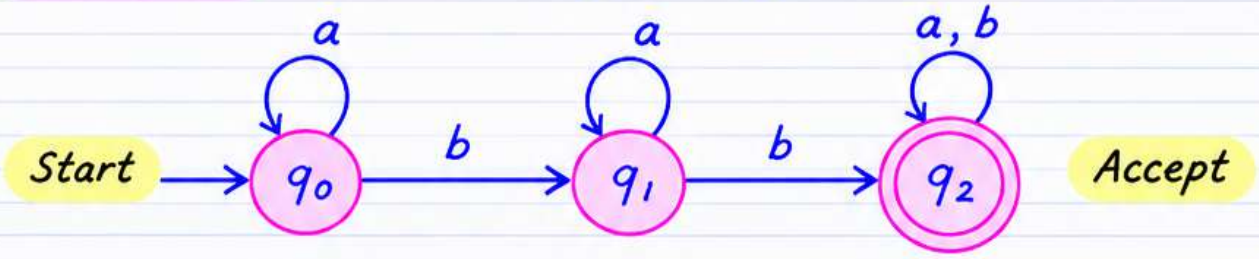
- **Definition :**

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA. A string w is rejected by the machine M if, after processing w from the initial state q_0, the machine does not reach a final state. That is, $\delta^*(q_0, w) \notin F$.

$$L(M) = \{ w \in \Sigma^* \mid \delta^*(q_0, w) \notin F \}$$

$L(M)$ is the language rejected by DFA M.

- **Example :** Consider the following DFA :



- **Processing the input string abaa :**

Input : a b a a

Transitions :

$$\delta(q_0, a) = q_0$$
$$\delta(q_0, b) = q_1$$
$$\delta(q_1, a) = q_1$$
$$\delta(q_1, b) = q_2$$

State Path :



After processing the string abaa, the DFA reaches state q_1, which is not a final state. Therefore, the string abaa is **REJECTED** by the machine.

1.6 DFA Design Techniques

Pattern Recognition Problem

Q : What are the general steps to be followed while designing a DFA for a pattern recognition problem ?

Ans : There are 5 steps :

1) Identify the minimum string.
2) Identify the alphabet and construct a skeleton DFA.
3) Identify other transitions not defined in Step 2.
4) Identify the transitions not defined in Step 3.
5) Construct the complete DFA using the transitions identified in Steps 3 and 4.

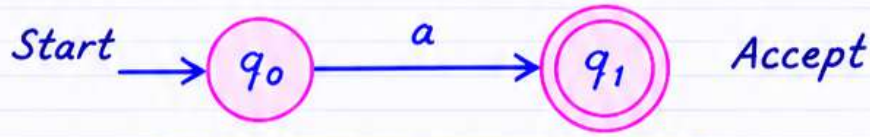
Example :

Design a DFA to accept strings consisting of one or more a's (at least one a).

Step 1 : Minimum string = a.

Step 2 : Alphabet : $\Sigma = \{a\}$.

Step 3 : Draw a skeleton DFA for the minimum string a :

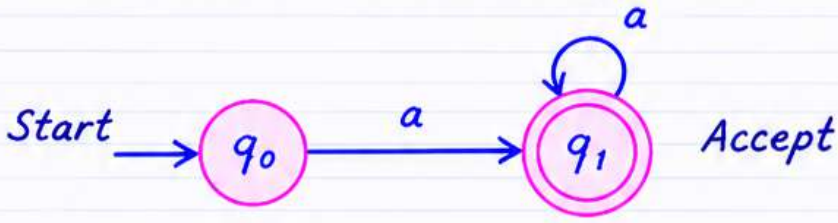


Skeleton DFA accepting a

Step 4 : Identify the missing transition : $\delta(q_1, a) = ?$

Step 5 : Complete the DFA by adding a self-loop on q_1 :

$$\delta(q_1, a) = q_1.$$

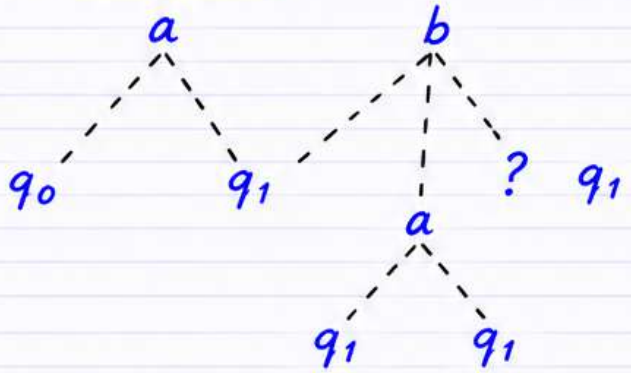


Here, q_0 has no other input symbol because $\Sigma = \{a\}$.

Conclusion : The DFA accepts a, aa, aaa, ... — all strings containing at least one a.

Step 5 : Construct the Final DFA

- **Transition Tree :**



$\delta(q_0, a) = q_1$

$\delta(q_1, a) = q_1$

- **Step 5 :** Construct the DFA using transitions in Step 3 and Step 4. The DFA is defined as,

$$M = (Q, \Sigma, \delta, q_0, F)$$

where,

$Q = \{q_0, q_1\}$

$\Sigma = \{a, b\}$

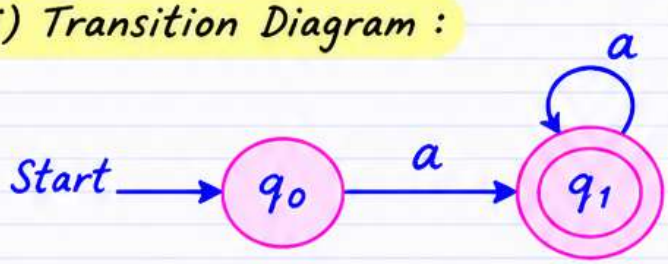
q_0 = start state

F = final state = $\{q_1\}$

δ = transition diagram and transition table shown below.

- **(i) State Set :** $Q = \{q_0, q_1\}$

- **(ii) Transition Diagram :**



(DFA for accepting a)

- **(iii) Transition Table :**

δ	a	b
→ q_0	q_1	q_1
* q_1	q_1	q_1

→ Transition from q_0

→ Transition from q_1

- **Conclusion :** The DFA accepts all strings containing at least one 'a'.

DFA Design Example – Language Accepted by DFA

- The language to be accepted by DFA can be written as follows:

$$L = \{ a, aa, aaa, \ldots \}$$

or

$$L = \{ a^n \mid n \geq 1 \}$$

$a^1 = a$

$a^2 = aa$

$a^3 = aaa$

$a^4 = aaaa$

and so on.

- n is a positive integer (1, 2, 3, ...).